



MEMORIA DE PROYECTO: PACHANGAYA



Hecho por Manuel Bono

Módulo: Entornos de Desarrollo

Unidad: Módulo 2 – Reconocimiento de Elementos en el Desarrollo de un Programa Informático

Proyecto: PachangaYa (Organizador de partidos y buscador de suplentes)



ÍNDICE DE CONTENIDOS

1. Documentación Inicial

- 1.1. Nombre de la aplicación
- 1.2. Descripción breve
- 1.3. Objetivos (qué problema resuelve)
- 1.4. Usuarios principales (roles)
- 1.5. Plataformas
- 1.6. Modelo de desarrollo sugerido

2. Selección y Justificación de la Aplicación

- 2.1. Conexión con los conceptos de clase
- 2.2. Por qué es una aplicación realista

3. Listado de Características del Sistema

- 3.1. Tabla de las 10 características principales

4. Análisis Detallado de Requisitos

- 4.1. Requisitos Funcionales (RF)
- 4.2. Requisitos No Funcionales (RNF)

5. Reflexión sobre las Fases del Desarrollo y Modelos

- 5.1. La fase más difícil del proyecto
- 5.2. Comparativa entre modelos de desarrollo



1. DOCUMENTACIÓN INICIAL

1.1. Nombre de la aplicación

PachangaYa

1.2. Descripción breve

PachangaYa es una aplicación para el teléfono pensada para organizar partidos de fútbol, baloncesto o pádel entre amigos en menos de un minuto. En lugar de mandar cientos de mensajes en grupos donde nadie se aclara, la aplicación te dice cuántos jugadores están listos para jugar, quién falta y, si alguien avisa a última hora de que no puede ir, encuentra un sustituto que viva cerca.

1.3. Objetivos (qué problema resuelve)

- **El lío de organizar:** Evita las discusiones en grupos de mensajes para saber a qué hora se juega y quién va seguro.
- **El problema de la persona que falta:** Si alguien cancela media hora antes de empezar, avisa a jugadores que estén libres para no tener que suspender el partido.
- **Hacer equipos justos:** Reparte a los jugadores para que los dos equipos tengan un nivel parecido y nadie pierda siempre.
- **Saber llegar al campo:** Muestra en un mapa dónde está la pista para que nadie llegue tarde.

1.4. Usuarios principales (roles)

- **El Capitán (Organizador):** Es la persona que crea el partido, elige la pista, pone la hora y decide si el partido sigue adelante.
- **El Jugador:** Es el amigo que entra, pulsa "Voy", elige si quiere jugar de portero o de delantero y confirma que irá.
- **El Suplente (Jugador de reserva):** Es alguien que no tenía partido hoy, pero tiene la app lista para recibir avisos si a algún grupo cercano le falta una persona para jugar.

1.5. Plataformas

- **Aplicación Móvil (Android e iOS):** Es la pantalla principal, pensada para llevarla en el bolsillo y consultar los avisos mientras estás en la calle o en el patio.
- **Página Web Rápida:** Una web sencilla para que un amigo pueda confirmar que va usando un enlace, sin tener que descargarse nada si tiene el móvil lleno.

1.6. Modelo de desarrollo sugerido

Se elige el **Modelo Ágil (Scrum)** trabajando en pequeños pasos de dos semanas:

- **¿Qué significa esto de forma sencilla?**
- Es como construir con piezas de LEGO: en vez de intentar hacer una nave espacial gigante de golpe sin enseñársela a nadie durante un año, construimos primero una base con ruedas en dos semanas. Dejamos que nuestros amigos jueguen con ella, vemos qué partes se rompen o no les gustan, y en las siguientes dos semanas le añadimos las alas y los motores.
- **Por qué no usamos el modelo en Cascada:**
- En el modelo en Cascada tendríamos que escribir en un papel todas las pantallas del tirón sin poder cambiar nada. Si tras seis meses programando descubrimos que a la gente le parece aburrida, ya no habría marcha atrás y habríamos perdido todo el trabajo.



2. SELECCIÓN Y JUSTIFICACIÓN DE LA APLICACIÓN

Elegimos **PachangaYa** porque aplica de forma directa los principios aprendidos en clase de Ingeniería del Software:

1. **Entregar valor rápido (Filosofía Ágil):**
2. En clase aprendimos que el software no debe tardar años en ser útil. Con esta aplicación podemos crear primero una versión simple con una sola lista para apuntarse, probarla con amigos un fin de semana y comprobar si resuelve el problema antes de programar funciones más complejas.
3. **Definir bien a los usuarios (Roles):**
4. No todo el mundo necesita los mismos botones. En clase vimos que el diseño debe adaptarse al rol: el Capitán necesita botones para cambiar la hora o borrar a alguien que no viene, mientras que el Jugador solo quiere un botón gigante que diga "Voy" para no perder tiempo.
5. **Elegir la plataforma según el momento de uso:**
6. Un programa para jugar al fútbol con amigos no tiene sentido si solo funciona en un ordenador de escritorio grande. Como vimos al estudiar la arquitectura, el software debe vivir en el móvil porque las dudas y las bajas ocurren cuando ya estás de camino a la pista.

3. LISTADO DE CARACTERÍSTICAS DEL SISTEMA

Para entender la diferencia:

- **Requisito Funcional (RF):** Es lo que el programa **hace** (los botones y las acciones que puedes tocar, como marcar un gol o apuntarte a una lista).
- **Requisito No Funcional (RNF):** Es cómo de **bien, rápido o seguro** funciona el programa por dentro (como la velocidad de un coche o la fuerza de un escudo).

3.1. Tabla de las 10 características principales

N.º	Característica	Tipo	Explicación sencilla
01	Botón de apuntarse y borrarse	Funcional	El usuario toca un botón y su nombre aparece o desaparece de la lista del partido.
02	Aviso de emergencia "Falta 1"	Funcional	Si alguien se cae a última hora, la app avisa a otros jugadores disponibles por la zona.
03	Carga en menos de 1 segundo	No Funcional	La aplicación responde al instante para que nadie se canse de esperar la rueda de carga.



04	Creador de equipos equilibrados	Funcional	Mezcla a los 10 jugadores apuntados para formar dos bandos parejos y divertidos.
05	Pantallas fáciles de usar	No Funcional	Todo se entiende a la primera con dibujos grandes y sin tener que leer instrucciones.
06	Teléfonos y datos privados ocultos	No Funcional	Nadie que tú no conozcas puede ver tu número de teléfono ni tus datos personales.
07	Chat privado para el partido	Funcional	Un espacio para que los que van a jugar hablen de qué color de camiseta llevar.
08	Aguanta a mucha gente a la vez	No Funcional	El servidor no se apaga ni se rompe aunque 20 amigos entren a mirar la lista al mismo tiempo.
09	Mapa para saber cómo llegar	Funcional	Un botón que abre el mapa del móvil y te guía directo hasta la puerta de la pista.
10	Funciona en teléfonos viejos y nuevos	No Funcional	Cualquier amigo puede instalar la app, tenga el móvil que tenga.

4. ANÁLISIS DETALLADO DE REQUISITOS

4.1. Requisitos Funcionales (Lo que la aplicación hace)

- [RF-01] Crear y configurar un partido:
 - *Qué hace:* El Capitán puede crear una ficha eligiendo el deporte (fútbol, baloncesto, pádel), el día, la hora, el campo y cuántas personas hacen falta en total.
 -
 - *Por qué es importante (visto en clase):* Es la función principal del programa. Si la app no pudiera guardar esta información básica, ninguna de las demás funciones serviría para nada.
- [RF-02] Lista de espera automática:
 - *Qué hace:* Si las 10 plazas del partido ya están llenas y alguien más quiere entrar, la aplicación lo pone en una lista de espera. Si un titular avisa de que no puede ir, el primer suplente de la lista sube a titular solo.



- *Por qué es importante (visto en clase):* Aplica reglas automáticas dentro del programa, evitando las típicas peleas de "yo avisé antes por el grupo".
- **[RF-03] Botón de rescate para suplentes ("¡Falta 1!"):**
 - *Qué hace:* Cuando faltan pocas horas para empezar y no hay gente suficiente, el organizador pulsa un botón y la app busca a personas que vivan cerca y quieran jugar hoy.
 - *Por qué es importante (visto en clase):* Resuelve el problema principal por el que los partidos se suspenden en la vida real, aportando una solución directa al usuario.
- **[RF-04] Creador de dos equipos justos:**
 - *Qué hace:* El sistema reparte a los jugadores en Equipo A y Equipo B, asegurándose de que haya porteros en los dos lados y que no queden todos los mejores en el mismo equipo.
 - *Por qué es importante (visto en clase):* Añade valor práctico al programa, ya que ahorra los 15 minutos que los amigos suelen perder discutiendo cómo hacer los equipos antes de empezar.
- **[RF-05] Mapa con la ubicación de la pista:**
 - *Qué hace:* Muestra un mapa claro con el punto exacto del polideportivo o pista municipal donde se va a jugar.
 - *Por qué es importante (visto en clase):* En clase aprendimos que el software debe acompañar al usuario en todo su camino, asegurando no solo que se apunte, sino que llegue puntual al partido.

4.2. Requisitos No Funcionales (Cómo debe comportarse el sistema)

- **[RNF-01] Usabilidad (Confirmar asistencia en 2 toques):**
 - *Cómo debe ser:* La aplicación debe ser tan intuitiva que cualquier persona pueda abrirla, ver la hora y confirmar su plaza pulsando la pantalla solo dos veces.
 - *Por qué es importante (visto en clase):* La usabilidad mide lo fácil que es usar un programa. Si una app es enrevesada o pide rellenar muchos textos, la gente la borra y vuelve al método de siempre.
- **[RNF-02] Rendimiento (Menos de 1 segundo de espera):**
 - *Cómo debe ser:* Cada vez que tocas un botón para apuntarte o salirte, la pantalla debe actualizar la lista en menos de un segundo.
 - *Por qué es importante (visto en clase):* El rendimiento mide la velocidad del software. Si tarda mucho en responder, dos personas podrían pulsar a la vez sobre la última plaza libre y causar un error en los datos.
- **[RNF-03] Seguridad y Privacidad de los participantes:**
 - *Cómo debe ser:* Nadie en la app puede ver el número de teléfono privado de los demás jugadores; solo se ve el nombre y su foto de perfil.



- *Por qué es importante (visto en clase):* La seguridad de los datos personales es obligatoria por ley. Una aplicación debe proteger a sus usuarios para que confíen en ella y no sufran molestias de desconocidos.
- **[RNF-04] Fiabilidad (Disponible el 99,8% del tiempo):**
 - *Cómo debe ser:* Los servidores deben funcionar siempre bien, sobre todo por las tardes y los fines de semana, que es cuando todo el mundo juega al deporte.
 - *Por qué es importante (visto en clase):* La fiabilidad mide cuánto podemos confiar en que un programa no se rompa. Una app que se cae justo antes de empezar un partido deja a todo el mundo tirado.
- **[RNF-05] Portabilidad (Poco espacio y poca batería):**
 - *Cómo debe ser:* La app debe ocupar menos de 40 MB en la memoria del teléfono y gastar muy poca batería para que funcione sin problemas en móviles modernos y en modelos más antiguos.
 - *Por qué es importante (visto en clase):* La portabilidad garantiza que nadie del grupo de amigos se quede fuera del partido por tener un teléfono con poca memoria o más sencillo.

5. REFLEXIÓN SOBRE LAS FASES DEL DESARROLLO Y MODELOS

5.1. La fase más difícil del proyecto

La etapa que nos pareció más complicada de todo el trabajo fue el **Análisis de Requisitos**.

Al principio parece fácil decir simplemente "*vamos a hacer una app de fútbol*". Sin embargo, cuando te sientas a pensar qué debe ocurrir si dos personas tocan el botón a la vez, o cómo medir si la app es rápida sin decir solo "quiero que vuele", te das cuenta de que hay que poner números exactos (como decir "*menos de 1 segundo*"). En clase aprendimos que si los requisitos no se definen con claridad desde el primer día, los programadores construyen cosas que luego no funcionan como se esperaba.

5.2. Comparativa entre modelos de desarrollo

- **Por qué el modelo Ágil fue la mejor elección:**
 - En un proyecto para jugar al deporte entre amigos, la opinión de los usuarios cambia rápido. Con una metodología ágil como Scrum, podemos construir una pantalla pequeña, dejar que un grupo de amigos la pruebe este sábado y cambiar lo que no les guste la semana que viene.
- **Por qué el modelo en Cascada habría fallado:**
 - El modelo clásico en Cascada obliga a terminar y firmar todo el diseño antes de empezar a programar una sola línea. En una aplicación de este tipo, esperar al final para ver si a la gente le gusta el diseño habría sido un error muy caro y arriesgado.